

LIFEOPS — MASTER PROJECT PLAN v2

Fecha de inicio: 4 de agosto de 2026

Objetivo: construir un producto personal real y, simultáneamente, un proyecto de portafolio que demuestre .NET, React, mobile, SQL, Azure, Docker, microservicios, arquitectura hexagonal, mensajería, CI/CD e IA.

1. PRINCIPIO PRINCIPAL

LifeOps tiene dos objetivos igualmente importantes:

1. Ser una aplicación que realmente se utilice.
2. Servir como laboratorio práctico de ingeniería de software.

El proyecto NO debe convertirse en una colección de tecnologías agregadas únicamente para llenar un CV.

Cada tecnología debe resolver un problema concreto que el desarrollador sea capaz de explicar.

2. REGLA DE APRENDIZAJE

La IA/Codex actuará principalmente como:

- mentor;
- arquitecto consultivo;
- reviewer;
- profesor;
- debugger;
- entrevistador técnico.

Regla principal

El desarrollador escribe el código.

Codex no debe implementar automáticamente features completas.

Antes de desarrollar un concepto nuevo debe ocurrir este flujo:

1. Definir el problema.
2. Explicar el concepto.
3. Mostrar alternativas.
4. Explicar ventajas y desventajas.
5. Elegir una alternativa.
6. Justificar la decisión.
7. Definir los requisitos.

8. El desarrollador implementa.
9. Codex revisa.
10. El desarrollador corrige.

Código generado por IA

Por defecto:

NO generar implementaciones completas.

Se permiten:

- pseudocódigo;
- ejemplos pequeños;
- explicación de sintaxis;
- revisión;
- debugging;
- preguntas guía;
- diagramas;
- interfaces conceptuales.

La IA puede generar código repetitivo cuando el desarrollador ya demuestre entender claramente:

- qué está haciendo;
- por qué existe;
- cómo funciona;
- qué alternativas existen.

El criterio final será:

Si el desarrollador no puede explicar una parte de LifeOps durante una entrevista, esa parte todavía no está terminada.

3. VISIÓN DEL PRODUCTO

LifeOps

An AI-powered personal operating system for health, habits, finances, planning and career growth.

LifeOps centraliza información personal y la convierte en contexto útil para tomar decisiones.

Ejemplo:

Usuario:

¿Qué debería hacer hoy?

LifeOps puede considerar:

- agenda;
- hábitos pendientes;
- calorías;
- proteína;
- progreso de peso;
- sesiones de estudio;
- entrevistas;
- aplicaciones laborales;
- deudas;
- objetivos.

y producir una recomendación basada en datos reales.

4. ALCANCE DEL MVP

El MVP tendrá los siguientes dominios:

Identity

- registro;
- login;
- JWT;
- refresh tokens;
- perfil.

Habits

- crear hábitos;
- completar hábitos;
- streak;
- historial.

Health

- peso;
- objetivo;
- comidas;
- calorías;
- proteína;
- carbohidratos;
- grasas.

Career

- sesiones de estudio;
- preguntas de entrevista;

- respuestas;
- job applications.

Finance

- deudas;
- pagos;
- saldo pendiente.

Planning

- agenda diaria;
- tareas;
- prioridades.

AI

- chat;
- consulta de información;
- recomendaciones;
- tool calling.

Today

Dashboard central que combina información de todos los dominios.

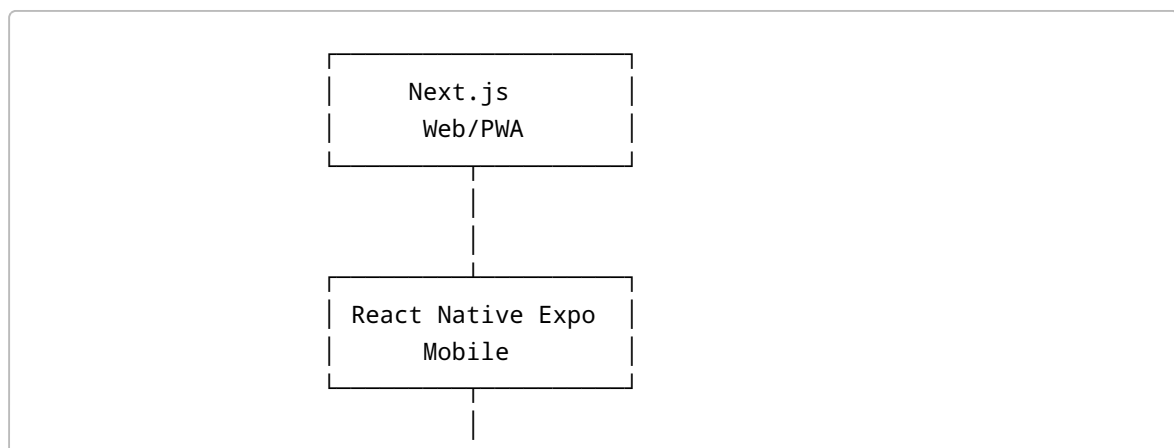
5. ARQUITECTURA OBJETIVO

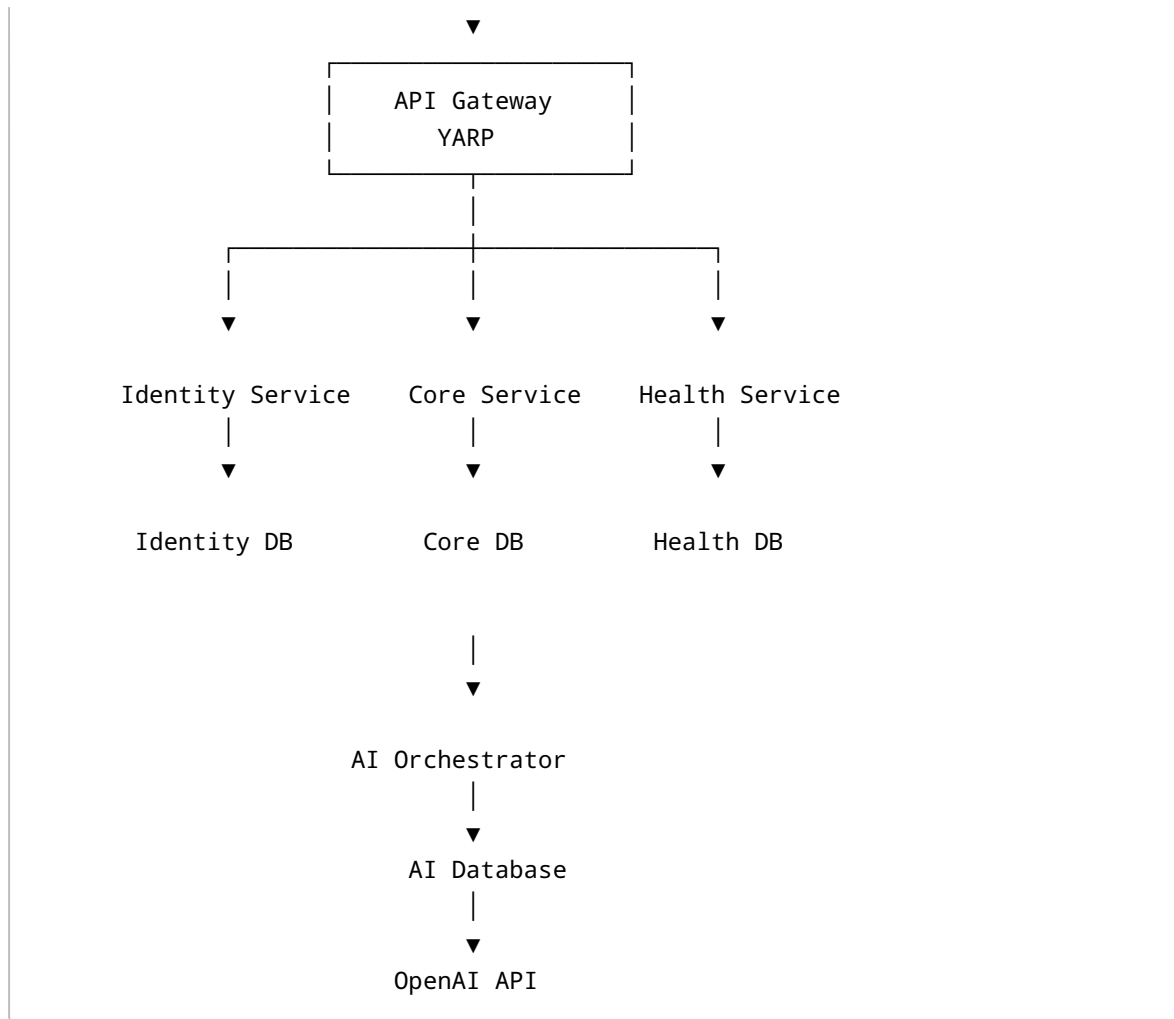
LifeOps utilizará:

Pragmatic Microservices + Hexagonal Architecture + Event-Driven Communication

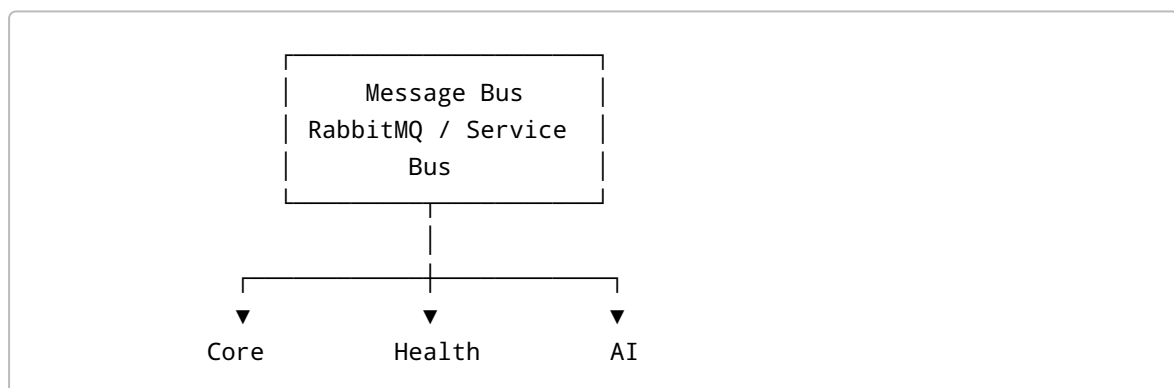
No se crearán veinte microservicios.

La arquitectura inicial tendrá solamente cuatro servicios backend.





Adicionalmente:



6. POR QUÉ MICROSERVICIOS

LifeOps podría construirse perfectamente como monolito.

La elección de microservicios es deliberada porque el proyecto también pretende enseñar:

- límites de servicios;
- comunicación distribuida;
- database per service;
- REST entre servicios;
- eventos;
- message brokers;
- eventual consistency;
- resiliencia;
- idempotencia;
- API Gateway;
- distributed tracing;
- despliegues independientes;
- CI/CD independiente.

La decisión NO se justificará simplemente con:

“Microservices scale better.”

La justificación real será:

LifeOps sirve como entorno controlado para aprender los problemas reales que introduce una arquitectura distribuida.

7. QUÉ ES HEXAGONAL EN LIFEOPS

Microservicios determina principalmente:

Cómo se divide y despliega el sistema.

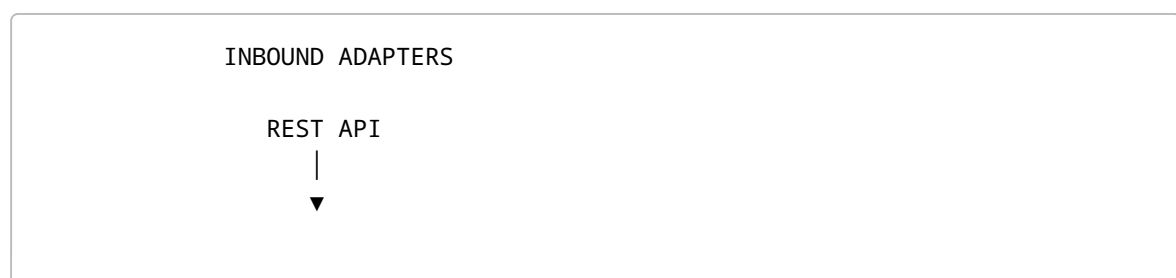
Hexagonal determina:

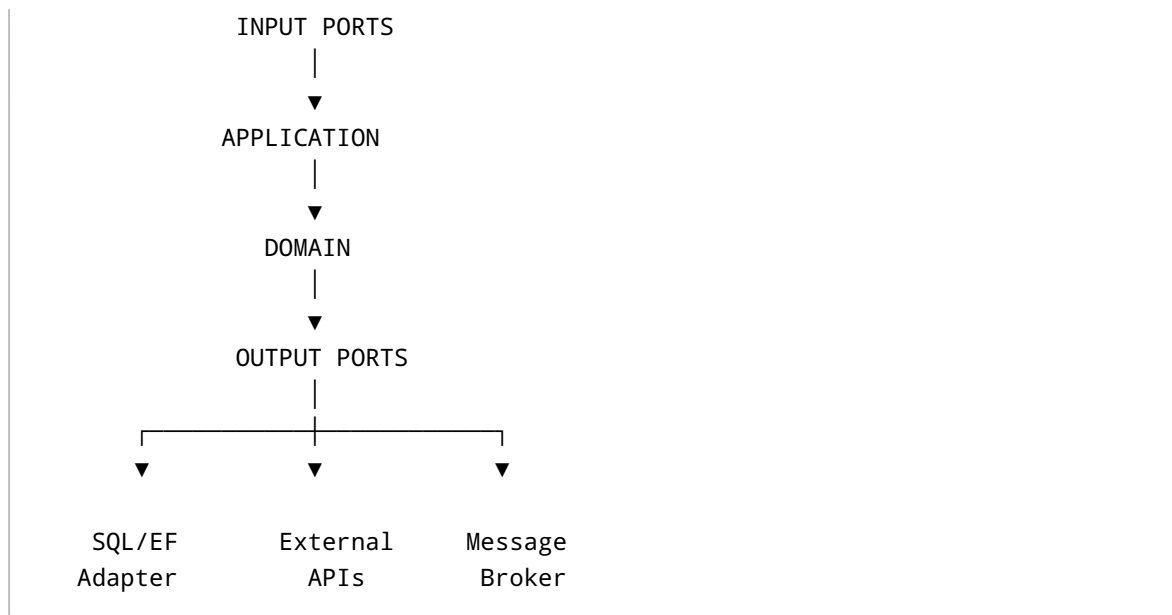
Cómo se organiza internamente cada servicio.

Cada microservicio importante utilizará principios de:

Hexagonal Architecture / Ports and Adapters.

Conceptualmente:





La lógica de negocio no debe depender directamente de:

- Entity Framework;
- OpenAI;
- Azure;
- RabbitMQ;
- USDA;
- HTTP.

Estas tecnologías serán adapters.

8. SERVICIO 1 — IDENTITY

Responsabilidad única:

Identidad y autenticación.

Incluye:

- User;
- Registration;
- Login;
- Password;
- JWT;
- RefreshToken;
- Profile básico.

Base de datos:

LifeOpsIdentityDb

Tecnologías:

- ASP.NET Core;
- ASP.NET Core Identity;
- EF Core;
- SQL Server.

No conoce:

- peso;
- comidas;
- hábitos;
- deudas;
- aplicaciones laborales.

9. SERVICIO 2 — CORE

Inicialmente agrupa:

```
Habits  
Goals  
Planning  
Finance  
Career
```

¿Por qué no un microservicio por cada módulo?

Porque:

```
HabitService  
GoalService  
DebtService  
PaymentService  
AgendaService  
CareerService  
JobService
```

sería distribución artificial.

Core contendrá varias capacidades relativamente pequeñas mientras el producto madura.

Base:

```
LifeOpsCoreDb
```


Posteriormente, si un dominio adquiere suficiente complejidad, podrá extraerse.

Ejemplo futuro:

```
Core
└─ Career
    ↓
Career Service independiente
```

Esto permitirá aprender también extracción de servicios.

10. SERVICIO 3 — HEALTH

Health sí tendrá un límite independiente desde el inicio.

Responsabilidades:

```
Weight
Measurements
Nutrition
Food
Meals
Macros
Recipes
```

Base:

```
LifeOpsHealthDb
```

Integraciones futuras:

```
USDA
Open Food Facts
Health Connect
Apple Health
```

Es un buen candidato a servicio independiente porque tiene:

- dominio propio;
 - datos propios;
 - APIs externas propias;
 - crecimiento potencial propio.
-

11. SERVICIO 4 — AI ORCHESTRATOR

Responsabilidades:

Conversation
AI context
Prompt management
Tool calling
Model routing
Token usage
Memory
AI permissions

Base:

LifeOpsAiDb

Integración:

OpenAI API

AI Service NO consulta directamente bases de otros servicios.

Debe consumir:

Core API
Health API
Identity context

o eventos/proyecciones cuando tenga sentido.

12. API GATEWAY

Tecnología:

YARP

Los clientes conocerán solamente:

api.lifeops.com

No:

```
identity.lifeops.com
core.lifeops.com
health.lifeops.com
ai.lifeops.com
```

Ejemplo conceptual:

```
/api/auth/*      → Identity
/api/core/*      → Core
/api/health/*    → Health
/api/ai/*        → AI
```

Temas a aprender:

- reverse proxy;
- routing;
- authentication;
- authorization;
- rate limiting;
- headers;
- correlation IDs.

13. DATABASE PER SERVICE

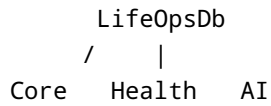
Regla arquitectónica:

Cada servicio es dueño de sus datos.

Correcto:

```
Identity → IdentityDb
Core → CoreDb
Health → HealthDb
AI → AiDb
```

Incorrecto:



Un servicio NO debe consultar directamente las tablas de otro.

Debe comunicarse mediante:

- API;
- eventos;
- proyecciones.

14. COMUNICACIÓN ENTRE SERVICIOS

Existirán dos mecanismos.

Síncrono

HTTP REST.

Ejemplo:

```

AI
↓
Health API
↓
GET daily nutrition
  
```

Usarlo cuando se necesita una respuesta inmediata.

Asíncrono

Eventos.

Ejemplo:

```

Health

WeightRegistered
↓
Message Bus
  
```

Usarlo cuando:

- no se necesita respuesta inmediata;
- otros servicios necesitan reaccionar;
- queremos desacoplamiento.

15. MESSAGE BROKER

Desarrollo local

Primera opción:

RabbitMQ + Docker.

Razones:

- gratis;
- muy usado;
- fácil de ejecutar localmente;
- permite aprender mensajería.

Conceptos:

Producer
Consumer
Queue
Exchange
Routing
Acknowledgement
Retry
Dead-letter queue

16. AZURE SERVICE BUS

Después de comprender RabbitMQ, se evaluará Azure Service Bus.

Objetivo:

entender diferencias entre:

Queue
Topic
Subscription

No migrar únicamente para agregar Azure al CV.

Primero debe existir una razón técnica.

17. EVENTUAL CONSISTENCY

LifeOps debe permitir experimentar deliberadamente con consistencia eventual.

Ejemplo:

HealthDb

Weight = 99.8 kg

Un dashboard basado en una proyección puede mostrar momentáneamente:

Weight = 100.1 kg

hasta procesar:

WeightUpdated

Esto permitirá aprender:

- eventual consistency;
 - idempotency;
 - duplicate messages;
 - retry;
 - outbox pattern posteriormente.
-

18. FRONTEND WEB

Stack:

Next.js
React
TypeScript

Objetivos de aprendizaje:

- components;
- hooks;
- routing;
- layouts;
- forms;
- state;
- server/client components;
- HTTP;
- authentication;
- error states.

Next.js NO sustituirá el backend .NET.

19. WEB COMO DESKTOP

LifeOps Web será también:

PWA

Por tanto podrá utilizarse en:

Browser

+

Desktop installed app

No crear todavía:

- Electron;
 - Tauri;
 - WPF;
 - MAUI Desktop.
-

20. MOBILE

Stack:

React Native
Expo
TypeScript

Android será la plataforma inicial.

Pantallas móviles MVP:

Login

Today

Quick Food Log

Habits

Agenda

AI Assistant

No es necesario replicar inmediatamente todas las pantallas administrativas de web.

21. TODAY DASHBOARD

Será la pantalla principal.

Ejemplo:

TODAY

HEALTH

Weight	100.4 kg
Calories	1300 / 2200
Protein	95 / 180

HABITS

- ✓ Gym
- React
- English

CAREER


```
Study today      45 min
Applications      3
```

FINANCE

```
Debt remaining    ...
```

AGENDA

```
09:00 ...
14:00 ...
```

ASK LIFEOPS

```
"What should I focus on?"
```

Today agrega información proveniente de varios servicios.

22. DECISIÓN IMPORTANTE SOBRE TODAY

No crear inicialmente un servicio llamado:

```
Dashboard Service
```

Primero el Gateway/frontend puede orquestar varias consultas.

Después podremos observar el problema.

Si aparecen demasiados requests:

```
GET /health/today
```

```
GET /core/habits/today
```

```
GET /core/agenda/today
```

```
GET /core/career/today
```

entonces podremos estudiar patrones como:

- Backend For Frontend;
- Aggregator;

- materialized view;
- CQRS read model.

Primero vivir el problema.

Después introducir el patrón.

23. NUTRITION

La IA no será fuente nutricional.

Pipeline:

User text

"2 huevos, arepa y queso"

↓

AI extraction

↓

Structured food query

↓

Nutrition provider

↓

Deterministic calculation

Fuentes:

Principal externa

USDA FoodData Central.

Fuente propia

CustomFoodCatalog.

Necesaria para alimentos regionales y recetas personales.

Posible futura

Open Food Facts.

24. LIFEOPS AI

El asistente trabajará mediante tools.

Ejemplos de lectura:

```
get_today_summary  
get_health_summary  
get_remaining_macros  
get_weight_history  
get_habits  
get_agenda  
get_debts  
get_job_applications  
get_study_history
```

Después:

```
log_weight  
log_food  
complete_habit  
create_task
```

25. SEGURIDAD DE AI

Reglas:

AI nunca recibe:

SQL credentials

AI nunca ejecuta:

raw SQL

Flujo correcto:

```
AI
↓
Tool
↓
Application Port
↓
Service API
↓
Domain
↓
Database
```

Operaciones destructivas o sensibles deberán requerir confirmación.

26. MEMORIA DE AI

Primera versión:

memoria estructurada.

```
UserProfile
Goal
Preference
Conversation
ConversationSummary
```

No introducir inicialmente:

```
Vector Database
Embeddings
Semantic Search
```

Cuando exista un problema real de recuperación de memoria, se evaluará RAG/vector search.

27. INFRAESTRUCTURA LOCAL

Durante desarrollo:

Docker Compose

- |— SQL Identity
- |— SQL Core
- |— SQL Health
- |— SQL AI
- |— RabbitMQ
- |— Identity Service
- |— Core Service
- |— Health Service
- |— AI Service
- |— Gateway

Pero esto NO será el Día 1.

Primero los servicios se ejecutarán manualmente.

Después se aprenderá Docker.

Finalmente se automatizará todo con Compose.

28. INFRAESTRUCTURA CLOUD

Propuesta:

Backend

Azure Container Apps.

Containers:

Gateway

Identity

Core

Health

AI

Database

Azure SQL.

Bases independientes:

IdentityDb

CoreDb

HealthDb

AiDb

Web

Vercel.

Mobile

Expo.

AI

OpenAI API.

CI/CD

GitHub Actions.

29. COSTOS

Objetivo durante fase personal:

GitHub	\$0
Vercel	\$0
Expo	\$0
Azure SQL	\$0 mientras entre en free tier
Container Apps	\$0 mientras entre en free allowance
RabbitMQ local	\$0

USDA	\$0
OpenAI	variable

Objetivo presupuestario inicial:

~\$5.000 - \$35.000 COP / mes

principalmente OpenAI.

30. COSTOS DE ENTRADA

Sin IA:

\$0

OpenAI:

aproximadamente:

US\$5 initial credit

Publicar Android:

US\$25 one time

Apple:

US\$99/year

Apple NO forma parte del MVP.

31. OBSERVABILIDAD

Primera etapa:

Structured Logging

Health Checks
Correlation IDs

Después:

OpenTelemetry

Y finalmente:

Application Insights

Objetivo:

seguir una request como:

Mobile
↓
Gateway
↓
AI
↓
Health
↓
SQL

y saber dónde falló o dónde fue lenta.

32. RESILIENCIA

Una arquitectura distribuida introduce fallos parciales.

Ejemplo:

Identity	UP
Core	UP
Health	DOWN
AI	UP

LifeOps no debería quedar completamente inutilizable.

Temas que se introducirán progresivamente:

Timeouts
CancellationToken
Retries
Circuit Breaker
Fallback
Health Checks

No implementarlos todos inmediatamente.

33. TESTING

Unit Tests

Lógica del dominio.

Integration Tests

API + Infrastructure.

Contract Testing

Posteriormente, para contratos entre servicios.

Stack inicial:

xUnit
WebApplicationFactory

Posteriormente:

Testcontainers

para SQL y RabbitMQ.

34. CI/CD

Cada servicio debe poder desplegarse independientemente.

Ejemplo:

```
health-service.yml  
  
core-service.yml  
  
identity-service.yml  
  
ai-service.yml
```

Pipeline conceptual:

```
Push  
↓  
Restore  
↓  
Build  
↓  
Tests  
↓  
Docker Build  
↓  
Container Registry  
↓  
Azure Container Apps
```

Pero CI/CD se implementará después de aprender el despliegue manual.

35. MONOREPO

Inicialmente:

```
LifeOps/  
  
apps/  
    web/  
    mobile/
```

```
services/  
  
  gateway/  
  
  identity/  
  
  core/  
  
  health/  
  
  ai/  
  
tests/  
  
docker/  
  
docs/  
  
  architecture/  
  
  adr/  
  
  diagrams/  
  
.github/  
  
  workflows/
```

Un solo repository.

No necesitamos cinco repositorios todavía.

36. ARCHITECTURE DECISION RECORDS

Cada decisión importante debe quedar documentada.

Inicialmente:

```
ADR-001  
Why Microservices for LifeOps  
  
ADR-002  
Service Boundaries
```

ADR-003
Hexagonal Architecture

ADR-004
Database per Service

ADR-005
Next.js for Web

ADR-006
React Native + Expo

ADR-007
YARP API Gateway

ADR-008
RabbitMQ for Development Messaging

ADR-009
Azure Container Apps

ADR-010
PWA Instead of Native Desktop

Cada ADR debe responder:

Context

Decision

Alternatives

Consequences

37. LO QUE NO ENTRA EN EL MVP

No:

Kubernetes

AKS

Kafka

Event Sourcing

Full CQRS

Service Mesh

GraphQL

Redis

gRPC

Vector DB

Temporal

Dapr

ElasticSearch

Multiple cloud providers

Separate repo per microservice

No porque sean malas tecnologías.

Porque todavía no existe un problema que las justifique.

38. CRONOGRAMA

FASE 0

4-5 de agosto

Architecture Audit

Objetivo:

entender lo existente.

Revisar:

Repository

Current entities

Database

Debt module

Habit module

Authentication

Existing architecture

Clasificar:

KEEP

REFACTOR

DELETE

LATER

Crear:

System Context Diagram

Service Boundary Diagram

ADR-001

ADR-002

No crear features.

39. FASE 1

6-9 de agosto

Backend fundamentals

Construir primero solamente:

Identity Service

Core Service

SIN Gateway.

SIN RabbitMQ.

SIN AI.

Objetivo:

entender:

ASP.NET Core API
Dependency Injection
EF Core
DTO
REST
Validation
Authentication
JWT
SQL

Resultado:

Client
↓
Identity
↓
IdentityDb

y:

Client
↓
Core
↓
CoreDb

40. FASE 2

10–13 de agosto

Hexagonal Architecture

Refactor consciente de uno de los servicios hacia Ports & Adapters.

Primero entender:

```
Domain  
  
Application  
  
Input Ports  
  
Output Ports  
  
Adapters  
  
Dependency inversion
```

Aplicarlo.

Después repetir el patrón en el segundo servicio.

Objetivo:

no aprender hexagonal copiando carpetas.

Aprender por qué existen las dependencias.

41. FASE 3

14–17 de agosto

Health Service

Crear:

Health Service

HealthDb

Features:

Weight

Goal Weight

Weight History

Después:

Nutrition

Primera integración externa:

USDA Adapter

Aquí hexagonal empieza a demostrar su utilidad.

42. FASE 4

18-20 de agosto

API Gateway

Introducir:

YARP

Antes:

Web → Identity

Web → Core

Web → Health

Después:

Web
↓
Gateway
↓
Services

Aprender:

Reverse Proxy

Routing

JWT propagation

Headers

Rate limiting concepts

43. FASE 5

21-24 de agosto

Web Application

Next.js + React + TypeScript.

Crear:

Login

Today

Habits

Weight

Nutrition

No buscar todavía diseño perfecto.

Objetivo:

end-to-end functionality.

44. FASE 6

25-27 de agosto

Career + Finance + Planning

Dentro de Core.

Career:

StudySession

InterviewQuestion

JobApplication

Finance:

Debt

DebtPayment

Planning:

AgendaItem

Mantener todos pequeños.

45. HITO — 27 DE AGOSTO

Para esta fecha LifeOps debería tener:

Identity Service

Core Service

Health Service

Gateway

3 databases

Next.js

Authentication

Habits

Health

Nutrition

Career

Debt

Agenda

Ya puedes ponerlo seriamente en CV como:

Full-stack personal operating system built with ASP.NET Core, React/Next.js, SQL Server and a distributed microservices architecture.

Aunque todavía falten messaging, AI y mobile.

46. FASE 7

28–31 de agosto

Docker

Aprender:

Image

Container

Dockerfile

Network

Volume

Environment variables

Containerizar:

Gateway

Identity

Core

Health

Después:

Docker Compose.

47. FASE 8

1-3 de septiembre

Messaging

Agregar:

RabbitMQ

No inventar eventos porque sí.

Elegir 1-2 casos reales.

Ejemplo:

StudySessionCompleted

o:

WeightRecorded

Aprender:

Producer

Consumer

Queue

Exchange

Acknowledgement

Retry

Idempotency

48. FASE 9

4-6 de septiembre

Distributed Systems

Provocar problemas deliberadamente.

Ejemplos:

Duplicate message

Health offline

Slow HTTP service

Message processing failure

Resolver:

Timeout

Retry

Idempotent consumer

Error handling

Estudiar:

Eventual consistency

49. FASE 10

7-10 de septiembre

AI Service

Antes de implementar estudiar:

LLM

Tokens

Context

Structured output

Tool calling

Prompt injection

Rate limits

Model cost

Crear:

AI Service

AI Db

OpenAI Adapter

Primero:

read-only tools.

get_today_summary

get_health_summary

get_habits

get_career_status

```
get_debt_status
```

Después:

una acción real.

Ejemplo:

```
complete_habit
```

50. FASE 11

11–13 de septiembre

Mobile

Expo + React Native.

Crear únicamente:

```
Login
```

```
Today
```

```
Habits
```

```
Nutrition quick log
```

```
AI Chat
```

No replicar inmediatamente toda la web.

51. FASE 12

14–17 de septiembre

Cloud + CI/CD

Desplegar manualmente primero.

Después automatizar.

Cloud:

Azure SQL

Azure Container Apps

Vercel

OpenAI

CI/CD:

GitHub Actions

52. FASE 13

18–20 de septiembre

Observability

Agregar:

Correlation ID

Health Checks

Structured Logging

Después:

OpenTelemetry

Ser capaz de seguir:

Browser

↓

Gateway

↓

AI

↓
Health
↓
SQL

53. FASE 14

21-23 de septiembre

Portfolio Polish

Crear:

README

System diagram

Service diagram

Sequence diagrams

ER diagrams

ADRs

Screenshots

Architecture explanation

Demo video

Technical challenges

Future roadmap

54. BÚSQUEDA LABORAL

No esperar al 23 de septiembre para comenzar a aplicar.

Objetivo:

Agosto

Construir.

Finales de agosto

Actualizar:

CV

LinkedIn

GitHub

Septiembre

Aplicar activamente mientras LifeOps continúa creciendo.

El proyecto puede aparecer como:

LifeOps — AI-Powered Personal Operating System

Tecnologías:

C#
ASP.NET Core
Microservices
Hexagonal Architecture
SQL Server
Entity Framework Core
React
Next.js
React Native
TypeScript
Docker
RabbitMQ
Azure
GitHub Actions
OpenAI API

Pero solamente agregar al CV tecnologías que realmente se hayan implementado y comprendido.

55. PLAN DE PREGUNTAS DE ENTREVISTA

LifeOps debe transformarse también en herramienta de preparación.

Cada concepto aprendido genera preguntas.

Ejemplo:

Después de EF Core:

IEnumerable vs IQueryable

Tracking vs NoTracking

N+1

Migrations

Transactions

Después de REST:

PUT vs PATCH

Idempotency

HTTP status codes

REST constraints

Después de microservices:

Database per service

Eventual consistency

Distributed transactions

Communication patterns

Service discovery

Después de RabbitMQ:

Queue vs pub/sub

Acknowledgement

Dead-letter queues

Delivery guarantees

LifeOps debe almacenar estas preguntas.

56. ALTERNATIVAS QUE DEBEMOS COMPARAR DURANTE EL PROYECTO

No simplemente aprender nuestra elección.

Comparar:

Architecture

Monolith
Modular Monolith
Microservices

Internal Architecture

Layered
Clean
Onion
Hexagonal
Vertical Slice

Communication

REST
gRPC
Messaging

Messaging

RabbitMQ
Azure Service Bus
Kafka

Database

SQL Server
PostgreSQL
MongoDB

Mobile

React Native
MAUI
Flutter
Capacitor

Hosting

Azure Container Apps
App Service
AKS
AWS

El objetivo no es dominar todas.

Es saber explicar por qué elegimos una.

57. REGLA ANTI-OVERENGINEERING

Antes de agregar cualquier tecnología preguntar:

1.

¿Qué problema existe actualmente?

2.

¿Por qué esta tecnología lo resuelve?

3.

¿Qué alternativa más sencilla existe?

4.

¿Qué costo introduce?

5.

¿Puedo explicar esta decisión?

Si no existe una respuesta clara:

BACKLOG

58. DEFINITION OF DONE — FEATURE

Una feature está terminada solamente cuando:

- ✓ funciona
- ✓ tiene manejo básico de errores
- ✓ está probada
- ✓ respeta boundaries
- ✓ el desarrollador entiende el código
- ✓ el desarrollador puede explicar decisiones
- ✓ no introduce dependencia arquitectónica incorrecta

59. DEFINITION OF DONE — LIFEOPS v1

Product

- ✓ Auth
- ✓ Today
- ✓ Habits
- ✓ Weight
- ✓ Nutrition
- ✓ Career
- ✓ Debt

- ✓ Planning
- ✓ AI Assistant

Platforms

- ✓ Web
- ✓ PWA/Desktop
- ✓ Android

Architecture

- ✓ Gateway
- ✓ Identity Service
- ✓ Core Service
- ✓ Health Service
- ✓ AI Service
- ✓ Database per Service

Distributed Systems

- ✓ REST between services
- ✓ RabbitMQ
- ✓ At least one event
- ✓ Idempotent consumer
- ✓ Resilience basics
- ✓ Eventual consistency understood

DevOps

- ✓ Docker

- ✓ Docker Compose
- ✓ Azure
- ✓ GitHub Actions
- ✓ CI/CD

Engineering

- ✓ Unit Tests
- ✓ Integration Tests
- ✓ Logging
- ✓ Health Checks
- ✓ OpenTelemetry basics

Portfolio

- ✓ README
- ✓ ADRs
- ✓ Diagrams
- ✓ Demo
- ✓ Architecture explanation

60. NORTH STAR

El objetivo no es construir la arquitectura más compleja posible.

El objetivo es llegar a poder decir:

"I designed and built a distributed full-stack application using ASP.NET Core microservices, Hexagonal Architecture, independent SQL databases, synchronous and asynchronous communication, React, React Native, Docker, Azure and an AI orchestration service."

Y cuando el entrevistador pregunte:

“Why?”

ser capaz de explicar cada decisión sin repetir frases memorizadas.

PRIMER PASO

NO crear todavía otro proyecto.

La primera tarea es:

LIFEOPS ARCHITECTURE AUDIT

Revisar el repositorio actual y producir:

1. Current repository structure
2. Existing projects
3. Existing database model
4. Existing features
5. Existing technical debt
6. Reusable code
7. Code that should be removed
8. Proposed migration path

Después crear:

ADR-001 – Why Microservices for LifeOps

ADR-002 – Initial Service Boundaries

Antes de escribir los nuevos servicios, el desarrollador debe poder responder:

¿Qué es un microservicio?

¿Qué diferencia tiene con un monolito?

¿Qué es un bounded context?

¿Por qué Health está separado de Core?

¿Por qué cada servicio tiene su base?

¿Qué problema resuelve un API Gateway?

¿Qué es hexagonal?

¿Qué es un port?

¿Qué es un adapter?

¿En qué dirección deben apuntar las dependencias?

Solo después comienza la implementación.